

Browser Networking

Why latency, not bandwidth, is the ceiling — what a protocol without multiplexing forces you to do about it, and the two ways to stop paying a round trip for every update.

1

Latency is the ceiling

The four delays, the speed of light, and the one you cannot buy your way out of.

2

What HTTP/1.x makes you do

Keepalive, pipelining, six connections, sharding, bundling — all workarounds.

3

Push, cheaply

Server-Sent Events: one long HTTP response, and what the browser does for free.

4

The full socket

WebSocket: both directions, binary, and everything the browser stops doing for you.

Print double-sided, short-edge bind, A4 · 100% scale (no “fit to page”).

Answer key starts at Section 9 — keep a sheet of paper over it.

How to use this booklet

The loop, per section

- 1. Read the plain-version box first. If it doesn't land, the rest won't either.
- 2. Read the teaching. Say each boundary out loud: “X is *this*, its look-alike Y is *that*.”
- 3. Cover the page. Write the answer to **Q1** in the blank lines, in your own words, in full sentences. **Then** check the key.
- 4. Score yourself on completeness. Then do **Q2** and note whether it beat Q1.
- 5. Only then move on.

THE RULE THAT DOES MOST OF THE WORK

If you find yourself thinking “yes, I know this” — that is *recognition*, and it is not learning. The only evidence that counts is a written answer produced with the page covered.

THE HABIT THIS SUBJECT REWARDS, SPECIFICALLY

Count round trips. Almost every claim in these pages resolves to a number of times a packet has to cross the network before something appears on screen, and almost every optimisation is an argument about reducing that count. When you meet a technique here, ask two questions: *how many round trips does it remove*, and *what does it cost me somewhere else*. Every one of them costs something somewhere else.

The spacing schedule

Review at **day 1, day 3, day 7, day 16, day 35**. On a pass, jump to the next interval. On a fail, reset that item to 1 day. Use the grid at the back; one row per item.

What's in here

1 — Learning goal and roadmap	orientation
2 — Latency is the ceiling	teaching · questions · matrix
3 — What HTTP/1.x makes you do	teaching · questions · matrix
4 — Push, cheaply: Server-Sent Events	teaching · questions · matrix
5 — The full socket: WebSocket	teaching · questions · matrix
6 — Concept sketch + master matrix	consolidation
7 — Symptom → diagnosis → move · reverse mapping	working reference
8 — Numbers worth remembering · Glossary	reference
9 — Answer key	keep covered
10 — Spaced-review tracker · final quiz	workbook

Learning goal & roadmap

LEARNING GOAL — WHAT YOU SHOULD BE ABLE TO DO WHEN YOU CLOSE THIS

Given a real performance question (“the page is slow, should we buy more bandwidth?”, “why did sharding make mobile worse?”, “should this feature use polling, SSE or a socket?”, “why does one big message stall all the others?”), you can:

1. **Name the delay** — propagation, transmission, processing or queuing — and say which of them the proposed fix actually touches.
2. **Count the round trips** before and after. Almost every argument in this subject is settled by that count.
3. **Prescribe the transport** — plain HTTP, SSE, or WebSocket — and name the near-miss you rejected and why.
4. **State the price.** Every technique here trades round trips against sockets, cache granularity, memory, battery, or the browser services you stop receiving.

The core model in one paragraph

Two numbers decide the performance of any network transfer: **bandwidth**, the maximum throughput of the path, and **latency**, the time for a packet to get from one end to the other. They are not symmetrical problems. Bandwidth is an engineering and money problem — lay more fibre, light more wavelengths — and it has been improving steadily. Latency is bounded by the speed of light, and we are already within a small constant factor of it, so the most a better cable can buy is a modest improvement. Everything else in this booklet follows from that asymmetry. A protocol that spends an extra round trip per resource is spending the one resource that cannot be manufactured, which is why HTTP/1.x forces a pile of workarounds — reuse the connection, open six of them, shard the domain, bundle the files — and why, when the server needs to send you something, the interesting question is whether you can avoid paying a fresh round trip to ask for it at all. Parts 3 and 4 are the two answers to that: one cheap and text-only, one that gives you the whole socket and charges you every service the browser used to provide.

TENTH-GRADER VERSION — THE WHOLE THING AT ONCE

- Two things decide how fast the network feels: how *wide* the pipe is, and how *long* it takes one thing to get to the other end.
- You can widen the pipe with money. You cannot make light faster — so distance is a hard floor.
- So the whole game is: ask fewer times, and don't wait for one answer before asking the next question.
- Old HTTP can only carry one answer at a time down a connection, so browsers open several and developers glue files together to ask less often.
- And if the server wants to tell you something, it either keeps one connection open and talks down it, or you open a proper two-way pipe — which is fast, but the browser then stops helping you with caching and compression.

The four parts, and why they're in this order

PART	WHAT IT COVERS	WHY HERE
1 Latency is the ceiling	Latency versus bandwidth; the four components of delay; the speed of light in fibre and the refractive index; last-mile latency; bufferbloat; bandwidth in the core versus at the edge; why one of the two is solvable and the other is not.	The floor everything else is built on. Without it, every optimisation in parts 2 to 4 looks arbitrary.

2 What HTTP/1.x makes you do	Keepalive and the $(N-1) \times RTT$ saving; pipelining and why it failed; head-of-line blocking; six connections per host and self-inflicted denial of service; domain sharding; protocol overhead and cookies; concatenation, spriting and inlining.	One missing protocol feature — multiplexing — generates every technique here. Seeing them as workarounds rather than best practices is the point.
3 Push, cheaply	The EventSource API; the event-stream wire format; what the browser handles for you, including automatic reconnect and last-seen message tracking; recovery strategies on the server; the two hard limits.	The first answer to “the server has something for me” that doesn't cost a round trip per update — and it is an ordinary HTTP response, so almost nothing has to change.
4 The full socket	The WebSocket API; the HTTP Upgrade handshake; frames versus messages and the binary framing layer; masking; subprotocol negotiation and extensions; head-of-line blocking again; proxies and timeouts; what you give up.	The most capable transport and the most expensive one. It only makes sense once you know exactly what parts 2 and 3 were buying you for free.

“We may not be able to make the packets travel faster, but we can reduce the distance by strategically positioning our servers closer to the users.”

1

PART 1 OF 4

Latency is the ceiling

Four delays, one of which you can buy your way out of, and one of which is physics.

TENTH-GRADER VERSION

- “Fast internet” is really two different things: how much you can send at once, and how long one thing takes to arrive.
- A wide pipe doesn't help if you have to wait for a reply before you can send the next thing. That waiting is latency.
- Some of the wait is because the signal has to physically travel. Light in glass does about 200,000 km per second, and you cannot argue with that.
- Some of it is because pushing bits onto a slow link takes time — that part *does* get better with a faster connection.
- And some of it is your packet sitting in a queue inside a router, which is the one nobody expects.

THE EVERYDAY VERSION

Posting a parcel. **Propagation delay** is how long the van takes to drive the distance — set by geography and the speed of the van, and the only way to shorten it is to move the warehouse closer. **Transmission delay** is how long it takes to load the parcels onto the van at the depot — a bigger loading bay genuinely fixes that, and it has nothing to do with how far the van is going. **Processing delay** is the few seconds a sorting office spends reading each label. **Queuing delay** is your parcel sitting in a full sorting office because more arrived than could be handled. Buying a bigger loading bay does nothing for the drive, and the drive is usually what you are waiting for.

The terms, each with its look-alike

TERM	PLAIN DEFINITION	CONCRETE EXAMPLE	BOUNDARY — WHAT IT GETS CONFUSED WITH
Latency	The time from the source sending a packet to the destination receiving it.	New York to London is 5,585 km: 28 ms one way in fibre, 56 ms for the round trip.	vs bandwidth , the maximum throughput of the path. They are different quantities with different economics, and conflating them is the single most common mistake in this subject. For most websites, latency is the bottleneck.
Propagation delay	Time for a message to travel from sender to receiver: distance ÷ signal speed.	Light in a vacuum is 299,792,458 m/s. In optical fibre the rule of thumb is 200,000,000 m/s , a refractive index of about 1.5.	vs transmission delay . Propagation is a function of distance and nothing else; transmission is a function of link rate and nothing else. This is the delay you cannot buy your way out of, and the only lever on it is shorter distance — which is what a content delivery network is for.

Transmission delay	Time to push all the packet's bits onto the link: packet length ÷ data rate .	A 10 Mb file takes 10 seconds to put on the wire over a 1 Mbps link and 0.1 seconds over a 100 Mbps link.	vs propagation again: this one has nothing to do with the distance between client and server. It is also the delay that more bandwidth genuinely fixes, which is why bandwidth upgrades help large transfers and barely help page loads.
Processing delay	Time for a router to examine the packet header, check for bit-level errors, and determine the outgoing route.	Much of this is now done in hardware, so the delays are very small — but they exist, and they are incurred at every hop .	vs queuing delay, which is time spent <i>waiting</i> to be processed rather than time spent <i>being</i> processed. More intermediate routers raise processing and transmission delay for each packet; heavier traffic raises queuing delay.
Queuing delay	Time an incoming packet waits in a router's buffer before it can be processed.	If packets arrive faster than the router can handle, they are buffered — and the wait is queuing delay.	Bufferbloat is this delay gone wrong: routers shipped with very large incoming buffers on the assumption that dropping packets should be avoided at all costs. That breaks TCP's congestion avoidance mechanisms and introduces high and variable latency. Active queue management — the CoDel algorithm — was proposed to address it.
Last-mile latency	The delay in the first few hops between you and your provider's core network.	One regulator's peak-hour measurements: 18 ms average for fibre-to-the-home, 26 ms for cable, 44 ms for DSL — just to reach the nearest measuring node.	vs the intuition that crossing oceans is the expensive part. It is often the last few miles, not the continents, where significant latency appears. When choosing a provider for browsing speed, low latency is worth optimising for.
Bandwidth at the edge	Available throughput is set by the lowest-capacity link anywhere between client and server.	Core fibre links carry hundreds of terabits per second — over 400 wavelengths multiplexed at up to 171 Gbit/s each, more than 70 Tbit/s per fibre. The edge is nothing like that.	vs assuming a fast link to your provider guarantees end-to-end performance. It does not: any intermediate node can be congested from demand, hardware failure or attack. High variability of both throughput and latency is an inherent property of these networks.
Great-circle path	The shortest route between two points on the globe — the best case for propagation.	New York to Sydney is 15,993 km, giving a 160 ms round trip in fibre along the great circle.	vs the real number. No such cable exists, so the packet takes a much longer route, and each extra hop adds routing, processing, queuing and transmission delay. The actual round trip is in the 200–300 ms range.

Why the two problems have different futures

- **Bandwidth is solvable.** Add more fibres to a link, deploy more links along congested routes, or improve the multiplexing to push more data through the fibre you already have. Between 2007 and 2011, more than half the added capacity on trans-Pacific cables came from **better equipment at both ends of existing fibre**, not new cable — and as of 2011 only about **20%** of deployed subsea capacity was in use. Every medium eventually reaches diminishing returns, but there is no fundamental reason throughput cannot keep increasing.
- **Latency is not.** Better materials with a lower refractive index and faster routers would help a little, but we are already within roughly 1.5× of the speed of light, so the most that strategy can deliver is a **modest ~30% improvement**. The alternative is to make the distance shorter, which means laying cable along the great circle — constrained by terrain, politics and cost.
- **So you design around it.** Reduce round trips, move the data closer to the client, and hide what remains behind caching and pre-fetching. Those are the only three moves, and everything in parts 2 to 4 is one of them.

THE NUMBERS THAT MAKE IT A HUMAN PROBLEM

Most people reliably report perceptible **lag** once a delay over **100–200 ms** is introduced. Past **300 ms** the interaction is described as sluggish. At **1,000 ms** many users have already performed a mental context switch — a daydream, or the next task. Set that against a 160 ms round trip to Sydney under ideal conditions and the budget becomes obvious: applications need to respond within hundreds of milliseconds, and the network does not leave much room for error.

CONCEPT BOUNDARY — “FASTER INTERNET” IS NOT ONE THING

More bandwidth reduces transmission delay. It helps most when you are moving a lot of bytes at once — a video stream, a large download.

It does not touch propagation delay, which is set by distance, so it barely helps a page that makes many small requests and waits for each answer. That page is spending round trips, not bytes.

The test: if you doubled the link speed, would the thing you are waiting for finish in half the time? If it wouldn't, you have a latency problem, and no amount of capacity will fix it.

Anchor sketch — the four delays, and which ones bend

Legend: bold = a named node

reversed = the node that matters most

Q = cover the page and answer this

total latency = the sum of four delays, at EVERY hop

PROPAGATION distance / speed of the signal

light in vacuum 299,792,458 m/s

light in FIBRE ~200,000,000 m/s (index ~1.5)

NY - London 5,585 km = 28 ms one way, 56 ms RTT

NY - Sydney 15,993 km = 160 ms RTT on the great circle; 200-300 ms on the cables that exist

+--> the ONLY lever is a shorter distance.
that is what a CDN is.

TRANSMISSION packet bits / link data rate

a 10 Mb file: 10 s at 1 Mbps, 0.1 s at 100 Mbps

nothing whatsoever to do with distance

+--> this is the delay more bandwidth fixes

PROCESSING read the header, check for errors, route
mostly hardware now: small, but once per hop

QUEUING waiting in a router's incoming buffer
big buffers look kind, break TCP congestion avoidance, and add HIGH and VARIABLE delay
= BUFFERBLOAT

bandwidth: more fibre, more links, better multiplexing.
an engineering and money problem.

latency: already within ~1.5x of light speed, so a better cable buys ~30% at most.
not a money problem. design around it.

Q Which two delays are functions of distance and link rate respectively, and which is a function of neither?

Q A colleague proposes upgrading the office link to fix a slow page. What do you ask first?

Q Why does a bigger router buffer make latency *worse*?

Trade-offs

DECISION	BUYS YOU	COSTS YOU	CHOOSE IT WHEN
More bandwidth	Lower transmission delay: large transfers finish sooner.	Money, and nothing at all for propagation delay — so a request-heavy page barely improves.	You are moving many bytes at once: video, downloads, backups.
A content delivery network	The only real lever on propagation delay — the data is physically closer, so every round trip is shorter.	Operational complexity, cost, and cache-invalidation discipline.	Almost always, and especially for globally distributed users.
Better fibre and faster routers	A little propagation delay and a little processing delay.	Capital expenditure for at most a ~30% improvement, because we are already close to the physical limit.	Rarely a lever available to you; know the ceiling so you can say why.

Laying a shorter cable	Genuinely less distance — the one thing that moves the floor.	Enormous cost, and constrained by terrain, politics and permission.	When milliseconds are worth millions. It is done, and it is priced accordingly.
Large router buffers	Fewer dropped packets.	Bufferbloat — TCP's congestion avoidance depends on drops as a signal, so hiding them produces high and variable delay.	Never as a default. Active queue management is the answer instead.
Designing around latency	Fewer round trips, data closer to the client, and the rest hidden behind caching and pre-fetching.	Application complexity — you are now doing work the network will not do for you.	Always. It is the only strategy that scales with distance.

Say it so you understand it → say it like an expert

SAY IT THIS WAY TO UNDERSTAND IT	SAY IT THIS WAY TO SOUND LIKE AN EXPERT
“The connection is slow.”	“Slow how — low throughput or high latency? They have different causes and different fixes, and for most pages it's latency.”
“Let's upgrade the bandwidth.”	“That reduces transmission delay. If we're waiting on round trips rather than bytes, it will barely move. Where's the time actually going?”
“We're on the other side of the world, that's just how it is.”	“Right — 200 to 300 milliseconds round trip in practice. We can't beat that, so we either reduce the number of round trips or move the data closer.”
“A CDN makes the site faster.”	“Specifically it cuts propagation delay by shortening the distance. It's the only lever we have on the one delay that isn't a money problem.”
“Our router has a big buffer so nothing gets dropped.”	“That's bufferbloat. TCP needs drops as a congestion signal — hiding them gives us high and variable latency instead.”
“The problem must be somewhere in the middle of the internet.”	“Check the last mile first. Tens of milliseconds can go before the packet even reaches the provider's core.”

PART 1 · QUESTION 1

A team's dashboard takes 4.2 seconds to become interactive for users in Sydney, against 1.1 seconds in London where the servers are. The page makes 78 requests. Someone proposes upgrading the origin's uplink from 1 Gbps to 10 Gbps, arguing that the origin is “clearly saturated” because Sydney is four times slower.

(a) Explain, in terms of the four delays, why the Sydney figure is what it is and why the uplink upgrade will not help. **(b)** Estimate the floor: state the round-trip time you would expect and the arithmetic that gets you there, and say what the 78 requests do to it. **(c)** Give the two structurally different fixes available, say which delay each attacks, and name the one thing that would genuinely lower the floor and why nobody does it.

PART 1 · QUESTION 2

An office reports that video calls became choppy and web browsing became erratic after the IT team replaced the gateway with a model advertising “zero packet loss under load”. Throughput tests still show the full contracted line speed. The team's conclusion is that the provider must be throttling.

(a) Name the most likely cause and explain the mechanism, including why the throughput test looks fine. **(b)** Explain precisely why a design goal of never dropping packets makes things worse rather than better. **(c)** Name the class of solution, and separately say why moving the video server closer would not fix this particular problem.

2×2 — what each delay depends on

COLUMNS: DOES IT DEPEND ON HOW *LOADED* THE PATH IS? →

	No — it is the same on an idle network	Yes — it grows with traffic
Does NOT depend on distance	Transmission and processing Pushing the bits onto the link, then reading the header and routing at each hop. <i>Move:</i> transmission is the one bandwidth genuinely fixes — 10 Mb takes 10 s at 1 Mbps and 0.1 s at 100 Mbps. Processing is mostly hardware now and small, but you pay it once per hop.	Queuing Your packet waiting in a router's incoming buffer because arrivals outpaced service. <i>Move:</i> this is where bufferbloat lives. Oversized buffers hide the drops TCP uses as a congestion signal, so you get high and <i>variable</i> delay instead of a clean backoff. Active queue management, not more buffer.
DEPENDS on distance	Propagation Distance divided by signal speed. In fibre, about 200,000 km per second. <i>Move:</i> there is exactly one lever, and it is not money — shorten the distance . That is what a CDN buys. Everything else in this booklet exists because this number cannot be reduced any other way.	The real path, not the great circle Longer routes mean more hops, and every extra hop brings its own processing, transmission and queue. <i>Move:</i> this is why the 160 ms great-circle figure for New York to Sydney is 200–300 ms in practice. Never quote the physics number as the expected number.

Read it as: the left column is what you get on a quiet network and the right is what load adds. The bottom-left is the focus because it is the only cell where spending money on capacity changes nothing at all — and it is usually the cell you are actually stuck in.

CARRY-OUT RULE FROM THIS PART

Before proposing a fix, say which delay it reduces. If the answer is “transmission” and you are waiting on round trips, you have bought nothing. Latency is not a capacity problem, and the only three moves against it are fewer round trips, shorter distance, and hiding it behind work done in advance.

What HTTP/1.x makes you do

One missing feature — multiplexing — and the pile of workarounds it generates.

TENTH-GRADER VERSION

- Setting up a connection costs a round trip before you've asked for anything. Reusing one connection instead of opening a new one for every file is the single biggest easy win.
- You'd think you could fire off all your questions at once. You can — but the answers must come back *in order*, so one slow answer holds up every answer behind it.
- Because of that, browsers just open six connections at once and hope.
- Developers then split their files across several hostnames to get more than six, and glue small files together to ask fewer times.
- None of that is clever. It is all patching around one thing the protocol cannot do: send two answers down one connection at the same time.

THE EVERYDAY VERSION

A serving hatch with one window. You can queue up and order one dish at a time, waiting at the hatch after each order — that is a connection without reuse, and you walk back and forth for every dish. Keepalive means you stay at the hatch. **Pipelining** means you shout all your orders at once, which is better — except the hatch is only wide enough to pass one plate at a time and they must come out in the order you asked. So if your first dish takes twenty minutes, the salad that was ready in one minute sits on the counter behind it. That is **head-of-line blocking**, and the restaurant's answer is not a wider hatch: it is opening six hatches.

The terms, each with its look-alike

TERM	PLAIN DEFINITION	CONCRETE EXAMPLE	BOUNDARY — WHAT IT GETS CONFUSED WITH
Keepalive (persistent connection)	Reusing one TCP connection for several HTTP requests. Two names, one feature.	Without it, every request costs two round trips — one for the handshake, one for request and response. With it, only the first does.	The saving for N requests is (N-1) × RTT , and the average page is now 90-plus resources, so the potential saving is measured in seconds. Enabled by default in HTTP 1.1. Watch your libraries: many default to non-keepalive because it gives a simpler API.
Pipelining	Dispatching requests without waiting for each response, moving the first-in-first-out queue from the client to the server.	Eliminates the idle round trip between responses, and lets the server process requests in parallel.	vs multiplexing , which HTTP 1.x does not have. Pipelining reorders <i>when requests are sent</i> ; it does not allow response data to be interleaved. That distinction is the whole of the next row.

Head-of-line blocking	HTTP 1.x requires strict serialisation of responses : each must be returned in full before the next one's bytes can be sent.	Client requests HTML (40 ms) and CSS (20 ms) together. The server finishes the CSS first — and must buffer it until the HTML is fully sent.	vs the TCP version of the same phrase, where a lost packet blocks every packet with a higher sequence number until it is retransmitted. Same name, different layer. The consequences here: underused links, server buffering costs, and unpredictable client latency.
Six connections per host	The browser's workaround for the absence of multiplexing: open several TCP sessions to the same origin.	Six parallel requests, six parallel server-side processes, and the initial congestion window effectively multiplied by six.	vs a solution. It buys parallelism at the cost of sockets, memory and CPU on client, server and every intermediary, plus streams competing for the same bandwidth. The number is a safe middle ground, not a calculation.
Self-inflicted denial of service	The connection limit protects the server — and can block your own application.	Open six parallel downloads to one host and issue a seventh: it hangs until one completes.	The trap: a WebSocket, an SSE stream or a hanging request each occupies a full TCP stream even while idle , because there is no multiplexing. Three long-lived connections leave you three ordinary requests.
Domain sharding	Splitting resources across several hostnames so the browser's per-host limit applies several times.	Shards are often CNAME records pointing at the same server: limits are enforced per hostname , not per IP address.	vs a free win. Each new hostname costs a DNS lookup, more sockets on both sides, and manual management of where resources live. Overused, it produces many connections that never escape TCP slow-start. There is no formula for the right number — it depends on the page and on each client's bandwidth and latency.
Protocol overhead	The HTTP metadata carried on every request, always in plain text, uncompressed.	500–800 bytes per browser-initiated request before cookies. A 15-character JSON body wrapped in 352 bytes of headers is 96% overhead.	vs the payload, which is often the smaller half. Headers are plain text for backwards compatibility; the standard sets no size limit, though many servers and proxies enforce 8 KB or 16 KB. Cookies are the usual largest offender and the usual forgotten one.
Concatenation · spriting	Combining several scripts or stylesheets into one resource; combining several images into one composite.	The byte delivery is the same as if pipelining worked — you have moved the pipelining logic up into your application.	vs free. All the bundled resources share one cache key, so a one-line change invalidates the lot; the bundle may contain code this page never needs; a sprite must be decoded and held in memory whole; and neither CSS nor JavaScript executes until the entire file has arrived.
Resource inlining	Embedding a resource in the document — script and style blocks, or the data: URI scheme for anything else.	Rule of thumb: consider it for resources under 1–2 KB , below which the HTTP overhead often exceeds the resource.	vs bundling. An inlined resource is part of the page , so it cannot be cached separately by the browser, a CDN or any proxy — inline it on ten pages and it ships ten times. Non-text assets need base64, which costs 33% byte expansion.

The arithmetic of a round trip saved

- The minimum cost of one HTTP request over a **new** TCP connection is **two network round trips**: one for the handshake, one for the request-response cycle. That is a fixed cost on every non-persistent session, on top of whatever

the server takes to produce the answer.

- **The faster your server, the more the fixed cost dominates.** Halving your server time does nothing to the two round trips.
- Worked example over a 28 ms one-way link, fetching an HTML document (40 ms of server time) and a CSS file (20 ms): **284 ms** over two separate connections; **228 ms** with keepalive, which removes one handshake; **172 ms** with pipelining, which removes an idle round trip between the two responses. That is a **40% reduction from protocol choices alone** — and the saving is not fixed. The higher the latency and the more requests, the larger it gets.
- One vendor reported a **3× improvement** for users on slower networks after making better use of existing connections through keepalive and pipelining in a desktop client.

Why pipelining failed, precisely

In theory the server could process pipelined requests in parallel and save another 20 ms. In practice, strict response serialisation means it cannot deliver them out of order, and that produces a list of problems the standard gives no guidance on:

- A single slow response blocks every request behind it.
- Servers processing in parallel must **buffer** the completed responses — and a very large response can exhaust server resources, which is an attack vector.
- A failed response may terminate the connection, forcing the client to re-request everything after it, which can duplicate processing.
- Detecting whether pipelining is safe is hard when intermediaries may be present: some abort the connection, others serialise everything.

Consequently browsers ship with it disabled or hidden behind an advanced setting. It remains genuinely useful **when you control both ends** — your client and your server, your retries and your idempotency, tunnelled over TLS so intermediaries cannot interfere.

CONCEPT BOUNDARY — THESE ARE WORKAROUNDS, NOT BEST PRACTICES

Two recommendations are evergreen and will be true under any protocol: eliminate unnecessary network latency, and minimise the bytes transferred. Fewer DNS lookups, fewer requests, a CDN, cache headers, compression, no redirects — all of these follow from those two.

Sharding, bundling, spriting and inlining are different. They exist to paper over one missing protocol feature. They are necessary today and you should know them cold, but they are not virtues, and each one buys its round trips with cache granularity, memory or complexity.

The tell: if a technique would become pointless the moment the protocol could interleave responses, it is a workaround.

Anchor sketch — one missing feature, five consequences

Legend: bold = a named node **reversed = focus node** Q = self-test cue

HTTP 1.x cannot MULTIPLEX: one response must be sent in full before the next one's bytes can go out

no keepalive -> 2 RTT per request (handshake, then request+response). a fixed cost.

KEEPALIVE -> 2 RTT for the first, 1 for every other. saving = $(N-1) \times \text{RTT}$, and the average page is 90+ resources.

PIPELINING -> send them all; the FIFO queue moves from client to server. saves another RTT, and allows parallel processing.

+--> **HEAD-OF-LINE BLOCKING** the CSS is ready first and must WAIT for the HTML to finish
+--> servers must buffer completed responses
+--> an attack vector
+--> a failed response can kill the connection
+--> intermediaries mangle or serialise it
= disabled in browsers. still excellent when you control both ends, tunnelled over TLS.

so the browser opens **6 CONNECTIONS PER HOST**

buys: 6 requests in flight, ~6x the initial cwnd
costs: sockets, memory, CPU, competing streams
and the 7th request simply waits -- which a long-lived stream will happily occupy forever

so you SHARD the domain for more

every shard = a DNS lookup + a fresh slow-start
no formula exists. hurts high-latency clients most.

so you CONCATENATE, SPRITE and INLINE

= pipelining moved up into your application
costs: cache granularity, memory, delayed execution

none of the last three are optimisations.
they are workarounds for ONE missing feature.

Q Give the keepalive saving as a formula, and say why the number of resources matters.

Q Pipelining sends requests in parallel. What exactly still cannot happen, and what four problems follow?

Q Name the cost of each of sharding, bundling and inlining — one distinct cost each.

Trade-offs

DECISION	BUYS YOU	COSTS YOU	CHOOSE IT WHEN
Keepalive	$(N-1) \times \text{RTT}$, no handshake per request, no repeated slow-start.	Essentially nothing. Idle connections hold resources, which is why intermediaries time them out.	Always. Check your libraries and proxies actually have it on.

Pipelining	Another round trip removed, and parallel processing on the server.	Head-of-line blocking, server buffering, fragile behaviour through intermediaries, and complex retry semantics.	Only when you control client and server, handle aborts and idempotency, and tunnel over TLS.
Six connections per host	Six requests in flight and a much larger effective initial congestion window.	Sockets and memory everywhere, competition between your own streams, and a hard ceiling that long-lived streams can consume.	You don't choose it — the browser does. Know what it implies for streaming transports.
Domain sharding	More parallelism than six.	A DNS lookup per shard, fresh slow-start per connection, and manual resource placement. Overuse actively slows things down.	Start at zero shards, add carefully, and measure. Very few sites benefit beyond about a dozen connections.
Concatenation and spriting	Fewer requests, less per-request overhead, and back-to-back delivery — pipelining at the application layer.	One cache key for everything, unused code shipped, whole-bundle invalidation on any change, sprite memory held in full, and execution delayed until the whole file lands.	Many small blocking resources. Consider splitting a stable core from volatile code.
Inlining	The request disappears entirely.	Uncacheable as a separate resource, duplicated on every page that inlines it, invalidating the host document on every change, and 33% expansion for binary assets.	Small, page-specific, rarely-changing resources — roughly under 1–2 KB.

Say it so you understand it → say it like an expert

SAY IT THIS WAY TO <i>UNDERSTAND IT</i>	SAY IT THIS WAY TO SOUND LIKE AN <i>EXPERT</i>
“We reuse the connection.”	“Keepalive, which saves (N-1) round trips across N requests — and with 90-odd resources on a page that's seconds, not milliseconds.”
“We send all the requests at once.”	“Pipelining moves the FIFO queue to the server. Responses still can't interleave, so one slow response blocks everything behind it.”
“Let's turn on pipelining in the browser.”	“It's disabled almost everywhere for good reasons — buffering, aborted connections, broken intermediaries. It's a win only when we own both ends and tunnel it.”
“Add more shards and it'll go faster.”	“Each shard costs a DNS lookup and a fresh slow-start. There's no formula — start at none, add carefully, and measure. Mobile clients suffer most.”
“We bundled everything into one file.”	“Which is application-layer pipelining. The bill is cache granularity: one change invalidates the whole bundle, and nothing executes until it all arrives.”
“We inlined the icons to save requests.”	“Then they're part of the document and can't be cached separately, and base64 added a third to their size. Fine under a kilobyte or two, expensive above it.”
“The seventh download just hangs.”	“Six connections per host, and our streaming connections are holding some of them. Without multiplexing an idle stream still occupies a full socket.”

PART 2 · QUESTION 1

A dashboard opens two Server-Sent Event streams and a long-lived polling request to `api.example.com`, then loads chart data from the same host. Users report that the fourth chart on the page never loads until they close a panel. A developer, seeing the streams are idle most of the time, argues they “cost nothing when no data is flowing” and proposes adding a fifth stream for notifications.

(a) Name what is happening and explain the mechanism, including why the streams being idle is irrelevant. **(b)** Say what the underlying protocol limitation is and why the browser's behaviour here is a deliberate safety feature that has backfired. **(c)** Give two structurally different fixes, and say what each one costs.

PART 2 · QUESTION 2

A team bundles all 40 of its JavaScript modules into one 900 KB file and all 120 of its icons into one sprite sheet. Desktop metrics improved. Mobile metrics got worse, repeat-visit performance got worse, and one engineer reports the app being killed on older handsets. The team's proposal is to shard the bundle across four subdomains so it downloads in parallel.

(a) Explain each of the three regressions in terms of what bundling actually trades away. **(b)** Say whether sharding addresses any of them, and justify the answer. **(c)** Give the changes you would make instead, and state what remains true regardless of protocol.

2×2 — fewer requests, or cheaper ones

COLUMNS: DOES IT MAKE EACH REQUEST *CHEAPER*? →

	No — the same cost per request	Yes — fewer round trips or bytes each
Does NOT reduce the NUMBER of requests	Run them at once instead Six connections per host; domain sharding. <i>Move:</i> understand that this changes neither the count nor the price — it only overlaps the waiting. And every extra connection costs a socket, and every extra shard costs a DNS lookup and a fresh slow-start. Start at zero shards and measure.	Keepalive and pipelining Same requests, fewer round trips: $(N-1) \times RTT$ from reuse, and one more from pipelining. <i>Move:</i> keepalive always. Pipelining only when you own both ends, because responses still cannot interleave. Also here: gzip, which cuts text transfers by 60–80% for one configuration flag.
REDUCES the number of requests	Bundling, spriting, inlining Fewer requests, and the bytes come back to back as if pipelining worked. <i>Move:</i> use it, and price it honestly. You pay in cache granularity, in memory for decoded sprites, in code this page never needed, and in execution that waits for the whole file. Inlining additionally costs 33% for anything binary.	Caching, and deleting A cache header that avoids the request entirely; an unused asset removed; a redirect that never happens. <i>Move:</i> this is the only cell that costs nothing elsewhere, and it is the one people skip. No request is faster than a request not made. Expires and ETag first, then fewer resources, then a CDN — and only then start rearranging the ones that remain.

Read it as: the top-left is where most tuning effort goes and where the least is gained. The two evergreen recommendations — eliminate unnecessary latency, minimise transferred bytes — both live in the bottom-right, and unlike everything else on this page they will still be true under any future protocol.

CARRY-OUT RULE FROM THIS PART

Ask whether the technique would survive multiplexing. Keepalive, caching, compression and deleting resources would. Sharding, bundling, spriting and inlining would not — they are compensation for a protocol that cannot interleave responses, and each one is paid for in cache granularity, memory or complexity.

Push, cheaply

Server-Sent Events: one HTTP response that never ends, and how much the browser does for you.

TENTH-GRADER VERSION

- Normally you ask and the server answers. If you want to know the moment something changes, asking over and over is wasteful and always slightly late.
- Instead the server answers once and simply never finishes: it keeps the connection open and writes new lines as things happen.
- The format is almost embarrassingly simple — lines beginning `data:` , separated by blank lines.
- The browser does the fiddly parts: it splits the messages, tells your code about each one, and if the connection drops it reconnects on its own and says which message it got last.
- Two catches: the server can talk, but you cannot talk back on it; and it is text only.

THE EVERYDAY VERSION

A live radio commentary versus phoning the ground every thirty seconds to ask the score. Phoning back repeatedly costs a call each time and you are always up to half a minute behind — that is polling. The commentary is one call that stays open, and you hear each goal the moment it happens. If the line drops, a good service dials back automatically and says “you last heard the 41st minute”, so nothing is missed. What you cannot do is talk back down the commentary line, and you cannot receive a photograph on it.

The terms, each with its look-alike

TERM	PLAIN DEFINITION	CONCRETE EXAMPLE	BOUNDARY — WHAT IT GETS CONFUSED WITH
Server-Sent Events	Efficient server-to-client streaming of text-based event data over a single long-lived HTTP connection.	Two components: the <code>EventSource</code> interface in the browser, and the event stream data format on the wire.	vs rolling your own streaming request. Under the hood SSE is an efficient cross-browser implementation of exactly that — the difference is that the browser handles connection management and message parsing, so your code holds only the business logic.
EventSource	The browser API: give it a URL, register callbacks, and receive each message as a DOM event.	<code>onopen</code> when established, <code>onerror</code> on failure, <code>onmessage</code> for untyped events, and <code>addEventListener</code> for a named event type.	vs a raw streaming request, which buffers the whole response until the connection drops. An SSE connection can discard processed messages instead of accumulating them, so there are no unbounded buffers . It can also stream from another origin using the same permission workflow as an ordinary cross-origin request.

Event stream format	The wire protocol: a streaming HTTP response with content type <code>text/event-stream</code> , carrying newline-delimited fields.	Four fields. <code>data:</code> — the payload, one or more adjacent lines. <code>event:</code> — an optional type. <code>id:</code> — an optional identifier. <code>retry:</code> — the reconnect interval, in milliseconds.	vs a framing protocol with lengths and types. There is no such thing here: event boundaries are marked by newlines , and the payload is the concatenation of the adjacent data fields. If an event has a type, a custom DOM event fires; otherwise the generic message callback is invoked.
Automatic reconnect	If the connection drops, the browser reestablishes it without any application code.	The specification recommends a 2–3 second delay, which is the common default, and the server can override it at any time by sending a <code>retry</code> field.	vs a reconnect you write yourself, which you would also have to get right for every browser. Note the corollary: the client cannot easily distinguish “quiet” from “dropped”, and on a mobile radio the connection is not dropped merely because the radio went idle.
Last-Event-ID	The browser remembers the ID of the last message it saw and sends it as an HTTP header on reconnect.	Message <code>id: 43</code> arrives, the connection drops, and the reconnect request carries <code>Last-Event-ID: 43</code> . The server resumes at 44.	vs a guarantee of delivery. The browser does the remembering and the asking; the recovery is entirely the server's job . If lost messages are acceptable, send no IDs and let the client resume. If they are not, assign IDs and keep a local cache to retransmit from. How far back to keep is your decision.
UTF-8 only	All event source data is UTF-8. SSE is not a mechanism for transferring binary payloads.	You can base64 a binary object to make it stream-friendly, at the usual 33% byte cost.	vs an oversight. It is a deliberate design decision: SSE is a simple, efficient, server-to-client transport for text. If you need binary, the right tool is a WebSocket. Note the mitigation — the stream is an ordinary HTTP response, so it can be gzip compressed like any other.
Server-to-client only	The client opens the connection and then receives. It cannot send data on it after the handshake.	So SSE does not address request streaming — uploading a large file as it is produced, for instance.	vs WebSocket, which is bidirectional. This is the first of SSE's two hard limits and the usual reason to reach past it.
SSE as a notification	Using the stream to say <i>that</i> something exists, and an ordinary request to fetch it.	The stream announces a new binary asset; the application dispatches a request to download it.	The trade is explicit: you pay one extra round trip , and you get back everything an ordinary HTTP request provides — response caching, transfer encoding, and the rest. A streamed asset cannot be cached by the browser cache at all.

What the wire actually looks like

The client sends an ordinary request with `Accept: text/event-stream`. The server replies `200` with `Content-Type: text/event-stream`, a keepalive connection and chunked transfer encoding — and then simply keeps writing. Everything after that is text:

- `retry: 15000` sets the client's reconnect delay to 15 seconds.
- A line of `data:` followed by a blank line is a complete message with no type — it triggers the generic message callback.
- Adding `event: foo` makes it fire as a custom DOM event of type `foo` instead.
- Adding `id: 42` makes the browser remember it, and send it back on the next reconnect.
- Two adjacent `data:` lines are **one** message with a newline in it, not two messages.

That is the entire protocol. It is trivial to implement on a server, and the reason it is worth learning as a format rather than an API is that the simplicity of the format is exactly what lets the browser take the work off you.

TWO DEPLOYMENT REALITIES WORTH KNOWING BEFORE YOU SHIP IT

Intermediaries. Proxies and firewalls that are not stream-aware may **buffer** the event stream, which turns into increased latency or an outright broken connection. The reliable answer is to deliver the stream over TLS, so nothing in the middle is in a position to buffer it.

Batteries. Real-time push, exactly like polling, can be expensive on a handset. Batch messages so the radio is woken less often, and do not add application-level keepalives you do not need — the connection is not dropped simply because the radio has gone idle.

CONCEPT BOUNDARY — LOW LATENCY HERE MEANS ONE SPECIFIC THING

It does not mean the packets travel faster. Switching from polling to a streaming transport does not change the round trip between client and server at all; that is propagation delay and it is set by physics and distance.

It means the queuing latency is gone. With polling, a message can be sitting ready on the server and still wait for the next client request — the delay is a function of your polling interval. With a persistent connection the server dispatches it the moment it exists.

Say it precisely: streaming eliminates *message queuing* latency, not *propagation* latency.

Anchor sketch — an HTTP response that never ends

Legend: bold = a named node **reversed = focus node** Q = self-test cue

an ordinary HTTP request whose response never finishes
-> GET /stream Accept: text/event-stream
<- 200 OK Content-Type: text/event-stream
 Transfer-Encoding: chunked
... and then the server just keeps writing

|
THE FORMAT four fields, newline-delimited
data: the payload. adjacent lines join into ONE
 message with a newline between them.
event: optional type -> a custom DOM event
id: optional. the browser REMEMBERS it.
retry: the reconnect delay, in milliseconds
(a blank line ends the event. that is all there is.)

|
WHAT THE BROWSER DOES for you, free
parses incrementally, discards processed messages
-> no unbounded buffer
fires a DOM event per message
reconnects on its own (2-3 s recommended default)
and sends **LAST-EVENT-ID** on the reconnect

|
WHAT YOU STILL OWE
the actual recovery: a server-side cache, IF lost
 messages matter. no IDs = no recovery, and
 that is a legitimate choice.
gzip on the stream
TLS, or middleboxes may buffer your stream
batching, or you will wake the radio all day

|
THE TWO HARD LIMITS
SERVER TO CLIENT ONLY -- no request streaming
UTF-8 ONLY -- binary costs 33% via base64
|
+--> so use it as a DOORBELL: stream the
 notification, fetch the asset with a normal
 request, and get caching back for one RTT

Q Name the four fields and say what a blank line does.

Q The browser sends the last-seen ID on reconnect. What does that guarantee, and what doesn't it?

Q Give both hard limits and the standard workaround for the second, including its cost.

Trade-offs

DECISION	BUYS YOU	COSTS YOU	CHOOSE IT WHEN
Polling instead	Simplicity, and no long-lived connection to keep alive through your infrastructure.	Message delay bounded by the polling interval, and full HTTP overhead on every poll — hundreds of bytes of headers for nothing much.	Updates are rare and lateness is genuinely free.
Server-Sent Events	Messages the moment they exist; as little as 5 bytes of overhead per message; browser-side parsing, reconnect and last-ID tracking.	A long-lived connection occupying one of six per host, text only, and one direction only.	Server-to-client text updates: notifications, feeds, progress, live figures.

Assigning event IDs	The client tells you where it left off, so you can retransmit what it missed.	You must implement and size a server-side cache, and decide how far back to keep.	Lost messages actually matter. If they don't, skip IDs entirely — it is a supported choice, not laziness.
Base64 over the stream	Binary data through a text-only transport.	33% byte expansion, and the asset can never be cached because it was streamed.	Rarely. Prefer the notification-then-fetch pattern, or a WebSocket.
Notification then fetch	Caching, compression and every other HTTP service, on the asset itself.	One extra round trip per asset.	Whenever the pushed thing is large, binary, or worth caching.
Streaming over TLS	Intermediaries can no longer buffer or mangle the stream.	Handshake cost on connect, and certificate management.	Any real deployment. Buffering middleboxes are the classic SSE failure.

Say it so you understand it → say it like an expert

SAY IT THIS WAY TO <i>UNDERSTAND</i> IT	SAY IT THIS WAY TO SOUND LIKE AN <i>EXPERT</i>
“The server pushes to the browser.”	“A single long-lived HTTP response with content type text/event-stream. The browser parses it incrementally and fires a DOM event per message.”
“Streaming is lower latency than polling.”	“It removes the queuing latency — the wait for the next poll. Propagation latency is identical; we haven't beaten the speed of light.”
“It reconnects automatically, so we won't lose messages.”	“It reconnects and reports the last ID it saw. Actually replaying what was missed is our server's job, and needs a cache.”
“We'll stream the thumbnails over it.”	“It's UTF-8 only — base64 would cost us a third, and a streamed asset can't be cached. Push a notification and fetch the image normally.”
“The stream works locally but dies in production.”	“Classic buffering intermediary. Put it over TLS so nothing in the middle can hold the bytes back.”
“Add a keepalive ping every ten seconds.”	“That wakes the radio on mobile for nothing. The connection isn't dropped because the radio went idle — check the infrastructure timeouts first.”

PART 3 · QUESTION 1

A trading desk streams price updates to browsers over Server-Sent Events. Every client reconnects roughly every 60 seconds, and after each reconnect traders see a gap in the tape. The stream also carries small chart images, base64-encoded into the data field, and the mobile app's battery consumption has become a complaint. The team's proposed fix is a heartbeat message every 5 seconds to “keep the connection from dropping”.

(a) Explain the gap: say what the browser does on reconnect and precisely where the responsibility for the missing messages lies. **(b)** Diagnose the 60-second reconnect cycle and say why the proposed heartbeat is the wrong fix and what it will cost. **(c)** Give the correct handling for the chart images, with the cost of your approach and two things it buys back.

PART 3 · QUESTION 2

A team is replacing 5-second polling with a push transport for a job-progress feature: the server emits short text status lines, and at the end a downloadable report. They are choosing between Server-Sent Events and a WebSocket. An architect argues for the WebSocket on the grounds that it “will be lower latency” and “we might want the client to send later”.

(a) Address the latency claim precisely, naming which latency changes and which does not. **(b)** Name three concrete things SSE gives them for free that they would have to build themselves on a socket. **(c)** Say how the final report should be delivered under your recommendation, and what would have to change for the WebSocket to become the right answer.

2×2 — who is talking, and in what

	No — text (UTF-8)	Yes — binary
Only the SERVER needs to send	Server-Sent Events Notifications, feeds, progress, live figures — anything the server knows first and expresses as text. <i>Move:</i> take it. About 5 bytes of overhead per message, gzip works on the whole stream, and the browser gives you parsing, reconnect and last-ID tracking for nothing. The cost is one of your six connections per host, and no way to talk back.	SSE as a doorbell The push announces that a binary asset exists; an ordinary request goes and gets it. <i>Move:</i> pay the extra round trip. In exchange the asset is cacheable by the browser and any intermediary, gets transfer encoding, and does not cost you the 33% that base64 would.
The CLIENT also needs to send	Depends how often Occasional client messages are ordinary requests — transactional, cacheable, authenticated, free. <i>Move:</i> don't reach for a socket because the client sends <i>sometimes</i>. SSE downward plus normal requests upward is a perfectly good design and keeps every browser service. Reach for a socket when upward messages are frequent and latency-sensitive.	WebSocket Both directions, either type, at any time — the only transport that does all of it. <i>Move:</i> Part 4. Note what the cell costs before you enter it: no caching, no automatic compression, no content type, and head-of-line blocking of your own making.

Read it as: the top-left cell is far larger than people expect, and the bottom-right far smaller. Most “we need a socket” requirements are really the top row plus the occasional ordinary request — and choosing the socket for those gives away caching, compression and metadata to buy something you were not going to use.

CARRY-OUT RULE FROM THIS PART

Ask what the client needs to send, and how often. If the answer is “nothing” or “occasionally, and it can be an ordinary request”, a one-way text stream plus normal requests will beat a socket on everything except upward latency — and will keep the caching, compression and reconnection logic you would otherwise be writing yourself.

The full socket

WebSocket: both directions, text and binary — and every service the browser stops providing.

TENTH-GRADER VERSION

- This is the closest thing to a plain two-way pipe the browser will give you. Either side can send anything, any time.
- It starts life as a normal web request that asks to be *upgraded*. Once the server agrees, HTTP is over and a different protocol takes the wire.
- Messages are chopped into frames with a tiny header — a handful of bytes instead of the several hundred an ordinary request carries.
- The catch is that nothing in the middle understands the pipe any more. No caching, no automatic compression, no labels on the data.
- And the old problem is back: two messages cannot share the pipe at the same time, so one big message stalls the small urgent ones behind it.

THE EVERYDAY VERSION

A dedicated phone line between two offices, installed by asking the public exchange for it and then never hanging up. It is fast, private and either side can speak at any moment. What you lose the moment it stops being a normal call is everything the public network was quietly doing: no operator who can put a common enquiry through to a recorded answer, no line that compresses your speech, no record of what kind of call it was. And the line carries one conversation at a time — start reading a long document down it and the urgent one-word message waits until you finish.

The terms, each with its look-alike

TERM	PLAIN DEFINITION	CONCRETE EXAMPLE	BOUNDARY — WHAT IT GETS CONFUSED WITH
WebSocket	Bidirectional, message-oriented streaming of text and binary data. The closest API to a raw network socket in the browser.	It is more than a socket: the browser adds connection negotiation, same-origin enforcement, interoperability with HTTP infrastructure, message framing, and subprotocol negotiation.	vs a replacement for the others. It is not one. The design intent is to layer <i>custom</i> application protocols — and the price of custom is that the application must supply the state management, compression and caching the browser otherwise provides.
The Upgrade handshake	The connection begins as an HTTP request carrying Connection: Upgrade and Upgrade: websocket , answered with 101 Switching Protocols .	Client sends Sec-WebSocket-Version and Sec-WebSocket-Key ; server must answer with Sec-WebSocket-Accept , a signed value derived from the key.	Why HTTP at all: it makes the protocol compatible with existing infrastructure and lets it run on ports 80 and 443, frequently the only open ports. After the handshake completes there is no further HTTP communication — the WebSocket protocol takes over the connection.

The key/accept exchange	An auto-generated client challenge, and a signed answer proving the server actually understands the protocol.	The key is concatenated with a fixed identifier from the standard, hashed, and base64-encoded back to the client.	vs authentication — it is neither. It is a security policy against blind upgrades by servers and intermediaries that would otherwise switch protocols without understanding what they had switched to. It does not protect you from a transparent proxy that quietly modifies data on the wire.
Frame vs message	A frame is the smallest unit on the wire: a variable-length header plus a payload that may be all or part of a message. A message is the complete sequence of frames making one logical application message.	Whether to split a message into frames is the implementation's decision. Your code never sees frames — the callback fires only when the <i>entire</i> message has arrived, even a 1 MB one.	vs a stream you must reassemble yourself. That is exactly what the framing layer removes. Header contents: a FIN bit marking the final fragment; a 4-bit opcode — text (1), binary (2), close (8), ping (9), pong (10); a mask bit; a variable-length payload length; and the masking key.
Framing overhead	The bytes each frame costs you.	Payload length is 0–125 inline; 126 means the next 2 bytes hold a 16-bit length; 127 means the next 8 bytes hold a 64-bit length.	Server-sent frames cost 2–10 bytes . Client frames add a 4-byte masking key, so 6–14 bytes . Compare with 500–800 bytes plus cookies for an HTTP 1.x request, and about 5 bytes for an SSE message. None of these include IP, TCP and TLS framing, which adds 60–100 bytes per message whatever the application protocol.
Masking	Every client-to-server frame has its payload masked with a 32-bit key carried in the header.	Server-to-client frames are not masked, which is why their overhead is 4 bytes lower.	vs encryption — it is not that either, and it protects nobody's confidentiality. It prevents a malicious script on the client from crafting bytes that perform a cache-poisoning attack on intermediaries that do not understand the protocol.
Subprotocol negotiation	The client advertises an array of application protocol names in the constructor; the server picks exactly one and returns it.	After onopen , the application reads the protocol attribute to see what was chosen.	vs content negotiation in HTTP. There is no per-message metadata at all here — one bit distinguishes text from binary and nothing else. So if the server supports none of the advertised protocols the handshake is not completed, the error callback fires, and the connection is terminated. Beyond selection, the chosen name has no effect on the API.
Protocol extensions	New opcodes and data fields that extend the framing layer itself, negotiated in the handshake, invisible to application code.	Two were developed: multiplexing , adding a channel ID so several logical connections share one TCP connection; and compression .	vs something you can rely on. As of the material here, multiplexing was in no popular browser, and compression support was limited to an older per-frame revision — suboptimal for large messages split across frames, and slated for deprecation in favour of per-message compression that was still experimental. Until that lands, compression is your application's job.

Head-of-line blocking (again)	Frames from different messages cannot be interleaved — there is no stream identifier in the framing layer.	A large message, even split into many frames, blocks delivery of frames belonging to other messages.	vs the HTTP 1.x version in Part 2 — same failure, one layer down, and now it is <i>your</i> message sizes causing it. <code>send()</code> is asynchronous and returns immediately, so a fast return is not delivery: check <code>bufferedAmount</code> , split large messages, and consider your own priority queue rather than dumping everything on the socket.
--------------------------------------	---	--	---

What you gave up when HTTP stopped

- **Caching, by anyone.** Stream an image over a socket and it will not be cached by the browser, by a CDN, or by any intermediary — so you may pay for the transfer repeatedly and drive far more traffic to origin. The same applies to video, text and everything else.
- **Compression.** An ordinary request negotiates its transfer encoding; an SSE stream is all text so the whole session can be compressed. A WebSocket carries both text and binary, and compressing a session wholesale makes no sense because the binary parts may already be compressed — so compression has to be per-message, and until the extension is widely available, that means your code.
- **Metadata.** There are no headers. If the receiver needs to know what a payload is, you must design that in: a fixed format agreed upfront, a consistent header inside the message, or a text message describing the binary message that follows.
- **Everything else the browser does with HTTP** — authentication, state management, range requests. The handshake *can* use existing cookies to validate the user and decline the upgrade, and that is the one HTTP service that survives.
- **The strategy that follows:** use the socket for what is genuinely non-cacheable — real-time updates and control messages — and let those messages trigger ordinary requests for anything an intermediary could usefully hold.

INFRASTRUCTURE WILL FIGHT YOU, AND IT IS USUALLY YOUR OWN

HTTP is optimised for short bursty transfers, so servers, proxies and load balancers are routinely configured to time out idle connections aggressively — precisely the wrong behaviour for a long-lived session. There are three layers: your own routers, balancers and proxies; external proxies you do not control; and the client's own network, which may block this traffic entirely and is the reason to keep a fallback. **TLS helps with the middle layer** by tunnelling past intermediaries, and in practice significantly raises the success rate of establishing the session — though it does not stop anyone timing out an idle TCP connection. For the layer you own, the timeouts must be raised explicitly: a common proxy defaults to **60 seconds**, and a common load balancer applies its connect, client and server timeouts only to the initial handshake, after which a separate **tunnel** timeout governs. Short timeouts are not stupidity — long idle sessions occupy memory and sockets everywhere — so before you add application keepalives, check the infrastructure.

CONCEPT BOUNDARY — A SOCKET IS NOT AN UPGRADE OVER THE OTHERS

It is not faster at the packet level. Switching transports does not decrease the round trip; propagation latency is identical. What it eliminates is queuing latency, in both directions rather than one.

It is not a superset. An ordinary request inherits caching, compression, authentication and range requests from a browser that understands the protocol. A socket is opaque by design, and the flexibility is the loss of those services — the same fact, described from two sides.

So it is not a default. Pick it when you need frequent, latency-sensitive, bidirectional or binary traffic, and route everything else over HTTP.

Anchor sketch — it starts as HTTP and then stops

Legend: **bold** = a named node **reversed** = focus node **Q** = self-test cue

```
-> GET /socket    Connection: Upgrade . Upgrade: websocket
    Sec-WebSocket-Version:    13
    Sec-WebSocket-Key:       an auto-generated challenge
    Sec-WebSocket-Protocol / -Extensions: what we speak
<- 101 Switching Protocols
    Sec-WebSocket-Accept:     the signed answer
    ... and from here there is NO more HTTP.
```

|

FRAME the unit on the wire. header + payload.
FIN . opcode . mask bit . length . payload
opcode: text 1 . binary 2 . close 8 . ping 9 . pong 10
length: 0-125 inline, 126 -> 2 more bytes,
 127 -> 8 more bytes
client frames are MASKED -- anti cache-poisoning,
aimed at intermediaries. NOT encryption.

MESSAGE the frames that make one application message.
your code never sees frames: onmessage fires
when the **WHOLE** message has arrived.

|

GAINED both directions, any time, text AND binary
2-10 bytes per server frame, 6-14 from the
client, vs 500-800 + cookies for HTTP 1.x
(all of it on 60-100 bytes of IP/TCP/TLS)

THE BROWSER STOPS HELPING

no caching -- not the browser, not a CDN
no automatic compression
no content-type, no headers, no metadata
HEAD-OF-LINE BLOCKING returns: frames from
different messages cannot interleave, and
send() returns long before anything is sent

v

so: the socket carries control and real-time updates,
and those updates **TRIGGER** ordinary HTTP requests
for anything worth caching.

Q Distinguish a frame from a message, and say which one your code sees.

Q Why are client frames masked, and what is it *not* protecting?

Q Name three services you stop receiving, and the design that gets them back.

Trade-offs

DECISION	BUYS YOU	COSTS YOU	CHOOSE IT WHEN
WebSocket over HTTP requests	Both directions at any moment, binary and text, and a handful of bytes per message instead of several hundred.	Caching, compression, metadata and most other browser services — plus a dedicated TCP connection out of your six per host.	Frequent, latency-sensitive, bidirectional traffic. Not for anything cacheable.
Secure WebSocket (over TLS)	A tunnel intermediaries cannot inspect, buffer or modify — and in practice a much higher success rate for establishing the session.	Handshake cost and certificate management. It still does not stop anyone timing out an idle connection.	Every real deployment, and especially mobile, where traffic crosses proxies that may not handle the protocol.
Subprotocol negotiation	Client and server agree, at connect time, on what the opaque bytes mean.	You must define and version the protocol yourself; if the server matches none of the advertised names, the connection is refused outright.	Always. It is the cheapest available protection against the two ends drifting apart.
A polyfill or real-time framework	One API across browsers, often with heartbeats, timeouts and automatic reconnection thrown in.	The fallback transport decides your performance, and the fallbacks are polling, long-polling or a streaming request — all worse. Some libraries need their own server.	When you must support clients without native support. Check what the fallback actually does before trusting it.
Large messages on one socket	Simplicity.	Head-of-line blocking: everything queued behind the big message waits, and <code>send()</code> returning is not delivery.	Never for latency-sensitive traffic. Split messages, watch the buffered amount, and prioritise.
Streaming assets over the socket	No protocol overhead on the bytes, and binary with no encoding penalty.	Nothing can cache it — not the browser, not a CDN — so you may re-transfer it endlessly and push all the load to origin.	Only for genuinely non-cacheable data. Otherwise send a notification and fetch it over HTTP.

Say it so you understand it → say it like an expert

SAY IT THIS WAY TO UNDERSTAND IT	SAY IT THIS WAY TO SOUND LIKE AN EXPERT
“It’s a socket in the browser.”	“Message-oriented and bidirectional, negotiated by an HTTP Upgrade so it works on ports 80 and 443 — and after the 101 there’s no HTTP left on the connection.”
“WebSocket is faster.”	“It removes queuing latency in both directions and cuts per-message overhead to a few bytes. Propagation latency is unchanged — the packets don’t move faster.”
“We’ll push the images down the socket.”	“Then nothing caches them — not the browser, not the CDN. Push the notification and let the client fetch them over HTTP.”
“send() returned, so it’s sent.”	“It’s queued. Check bufferedAmount — messages go in the order queued, so one big payload head-of-line blocks everything behind it.”
“Why is the payload masked?”	“Client-to-server only, and it’s not encryption — it stops a malicious script poisoning caches in intermediaries that don’t understand the protocol.”

“The connection drops after a minute in production.”	“Idle timeouts in our own proxy or balancer, most likely — and on a balancer the tunnel timeout is a separate setting from the handshake ones.”
“We’ll enable compression on the socket.”	“Per-message compression isn’t broadly available. Compress in the application, and optimise the binary payloads yourself.”

PART 4 · QUESTION 1

A collaborative map application sends cursor positions and edits over a WebSocket, and also streams map tiles down the same socket as binary messages because “binary is free on a socket”. Users report that cursors freeze for a second or two whenever the map is panned, and the operations team reports origin bandwidth roughly ten times what was forecast. The proposed fix is to increase the server’s send buffer.

(a) Name the mechanism behind the freezing cursors and say why a bigger send buffer will not fix it. **(b)** Explain the bandwidth figure — say precisely what stopped happening when the tiles moved onto the socket. **(c)** Give the redesign, name what each kind of traffic should travel over, and name one client-side check the application should be doing regardless.

PART 4 · QUESTION 2

A team ships a WebSocket feature. It works in development. In production, roughly a third of clients never establish a connection at all, and of those that do, many silently die after about a minute. Mobile users are worst affected. The team is on plain `ws://` behind a proxy and a load balancer they operate, and their fallback is a polyfill library whose default transport is polling.

(a) Give the most likely cause of the failed connections and the mechanism, and name the single change that addresses most of them. **(b)** Explain the one-minute deaths and say exactly where to look, including the setting people most often miss. **(c)** Say why the polyfill is a performance risk rather than a safety net, and what the handshake’s key-and-accept exchange does and does not protect against.

2×2 — the pipe, and who else can read it

COLUMNS: DOES THE CLIENT NEED TO SEND, FREQUENTLY? →

	No — the server does the talking	Yes — both ends, at any moment
Something in the MIDDLE should read it	Ordinary HTTP Anything a cache, a CDN or a proxy could usefully hold or transform: assets, documents, media. <i>Move:</i> leave it alone. Caching, compression, authentication and range requests all arrive free because the browser and every intermediary understand the protocol. This is where most bytes belong.	You are asking for both. Split it. Real-time interaction <i>and</i> cacheable payloads on one connection. <i>Move:</i> you cannot have both from one pipe — the properties are the same fact seen from two sides. Put control and updates on the socket, and let those messages trigger ordinary requests for anything worth caching. Paying one round trip buys back the browser, the CDN and the compressor.
Nothing in the middle NEEDS to read it	Server-Sent Events Text updates the server knows first and nobody else needs to interpret. <i>Move:</i> one long-lived HTTP response, about 5 bytes a message, and the browser handles parsing, reconnection and last-ID tracking. Cheaper than a socket in every sense that matters here.	WebSocket, properly used Control traffic, cursor positions, game state, custom binary protocols. <i>Move:</i> take it, and accept the bill knowingly — no caching, no automatic compression, no metadata, and head-of-line blocking you now manage yourself through message sizes and the buffered amount.

Read it as: the row is the question almost nobody asks and it decides more than the column. Every free service — a cache, a compressor, an authentication check — is a service that requires something in the middle to understand the payload. A private pipe is fast *because* nothing in the middle understands it, and that is also the entire bill.

CARRY-OUT RULE FROM THIS PART

Route by cacheability, not by excitement. The socket should carry what genuinely cannot be cached — updates, control, presence — and should trigger ordinary requests for everything else. A team that streams assets down the socket has bought a few bytes of framing and sold the browser cache, every CDN, and their origin bandwidth.

Concept sketch — the whole competency on one page

Three named groups, in the order the story runs: **FLOOR** → **WORKAROUNDS** → **PIPES**. Every node carries a question. Cover the answers, walk the map, and each answer hands you the next question.

Legend: **bold** = a named node **reversed = the one node to remember** **Q** = retrieval cue

The three groups run in order: **FLOOR** — what cannot be changed · **WORKAROUNDS** — what one missing feature costs · **PIPES** — the two ways to push.

GROUP 1 - FLOOR . the two numbers, and their economics

BANDWIDTH throughput. more fibre, more links, better multiplexing. a money problem, and improving.

LATENCY time to arrive. four delays:

- propagation distance / signal speed. fibre is ~200,000 km/s. ONLY fix: less distance.
- transmission bits / link rate. the one bandwidth fixes.
- processing header, checks, routing. once per hop.
- queuing waiting in a buffer. bufferbloat.

already within ~1.5x of light speed -> ~30% left, ever.
humans notice 100-200 ms, call 300 ms sluggish, and have mentally left by 1,000 ms.

v

GROUP 2 - WORKAROUNDS . HTTP 1.x cannot multiplex

keepalive -> saves (N-1) x RTT. N is 90+. always on.

pipelining -> saves one more RTT, then HEAD-OF-LINE BLOCKING: responses must be sent in full, in order. disabled in browsers.

6 per host -> parallelism, and a 7th request just waits. a long-lived stream holds one forever.

sharding -> more parallelism, at a DNS lookup and a fresh slow-start each. no formula exists.

bundling and -> fewer requests, paid for in cache

inlining granularity, memory and delayed execution.

evergreen: fewer round trips, fewer bytes. everything else here dies the day the protocol can interleave.

v

GROUP 3 - PIPES . when the server has something for you

SSE one HTTP response that never ends. text/event-stream.
data . event . id . retry, newline-delimited
the browser: parses, reconnects, resends LAST-EVENT-ID
you: the replay cache, gzip, TLS, batching
limits: server-to-client only, UTF-8 only

WEBSOCKET HTTP Upgrade -> 101 -> HTTP is over
frame (2-14 bytes) vs message (what your code sees)
gained: both directions, binary, tiny overhead
lost: caching, compression, metadata -- and head-of-line blocking is back

the rule: socket for control and real-time.
HTTP for anything worth caching.

The question per node — cover the right column

GROUP 1 — FLOOR

NODE	QUESTION	ANSWER IN ONE LINE
Latency vs bandwidth	Define both in one line each.	Latency is the time from sending a packet to receiving it; bandwidth is the maximum throughput of the path. Different quantities, different economics.
Reversed: propagation delay	Why is this the node to remember?	It is distance divided by signal speed, so no amount of capacity touches it. The only lever is a shorter distance — which is what a CDN is.
Transmission delay	What is it a function of, and what is it not?	Packet bits divided by link rate. Nothing to do with distance. 10 Mb takes 10 s at 1 Mbps and 0.1 s at 100 Mbps.
Bufferbloat	How does avoiding packet loss make things worse?	TCP uses drops as its congestion signal. Oversized buffers hide them, so you get high and variable delay instead of backoff.
Last mile	Where does the surprise latency live?	In the first few hops to the provider's core — tens of milliseconds before the packet is routed anywhere.
The ceiling	How much better can latency get?	We are within roughly $1.5\times$ of light speed, so better cable buys about 30% at most. Bandwidth has no such wall.

GROUP 2 — WORKAROUNDS

NODE	QUESTION	ANSWER IN ONE LINE
Keepalive	Give the saving as a formula.	$(N-1) \times \text{RTT}$ across N requests, because only the first pays the handshake round trip. With 90-plus resources that is seconds.
Head-of-line blocking	What exactly cannot happen in HTTP 1.x?	Response data cannot be interleaved: each response must be returned in full before the next one's bytes go out. A slow first response blocks all of them.
Why pipelining failed	Name three reasons.	Server buffering that can be attacked; a failed response killing the connection and forcing re-requests; and intermediaries that abort or serialise it.
Six per host	What is the hidden cost for streaming?	Without multiplexing, an idle long-lived stream still occupies a full connection — so a few of them starve ordinary requests.
Domain sharding	What does each shard cost?	A DNS lookup and a fresh TCP slow-start, plus manual management. There is no formula for the right number, and mobile suffers most.
Bundling and inlining	What are you really trading?	Requests for cache granularity: one change invalidates the bundle, sprites sit in memory whole, and inlined bytes ship with every page that inlines them.
The evergreen two	Which recommendations survive any protocol?	Eliminate unnecessary latency and minimise transferred bytes. Caching, compression, a CDN and deleting resources all follow from those.

GROUP 3 — PIPES

NODE	QUESTION	ANSWER IN ONE LINE
SSE on the wire	What is it, mechanically?	An ordinary HTTP response with content type text/event-stream that never finishes, carrying newline-delimited data, event, id and retry fields.

Last-Event-ID	What does the browser do, and what is left to you?	It remembers the last ID and sends it on reconnect. Replaying the missed messages is entirely the server's job, and needs a cache.
SSE's two limits	Name them and the workaround.	Server-to-client only, and UTF-8 only. For binary, push a notification and fetch over HTTP — one extra round trip, and caching comes back.
Queuing vs propagation	Which latency does streaming remove?	Queuing — the wait for the next poll. Propagation is unchanged; packets do not travel faster.
Frame vs message	Which does your code see?	The message. Frames are the wire unit with a 2–14 byte header; the callback fires only when the whole message has arrived.
Masking	What is it for, and what is it not?	Client-to-server frames only, to prevent cache-poisoning of intermediaries that do not understand the protocol. It is not encryption.
What a socket costs	Name three services you lose.	Caching by browser and CDN, automatic compression, and any payload metadata. Plus head-of-line blocking returns, now caused by your own message sizes.

Master 2x2 — the decision card you can carry to other problems

Two questions decide which pipe you need, and the second one is the one people skip.

COLUMNS: DOES THE OTHER END NEED TO TALK BACK, CONTINUOUSLY? →

	No — one direction is enough	Yes — both, at any moment
Something in the MIDDLE should be able to read it	Use the public road Every helper between the two ends — a cache, a compressor, a proxy, an auditor — can only help if it can read what is passing. <i>Move:</i> take the standard channel and accept its shape. You get work done for you by parties you never contracted with, which is the best price there is.	You are asking for both. Split the traffic. Real-time back-and-forth <i>and</i> payloads that others should be able to hold. <i>Move:</i> stop looking for one channel that does both — legibility and privacy are the same property seen from opposite sides. Run a fast private channel for the conversation, and let it point at the shared things, which travel the public road.
Nobody in the middle NEEDS to read it	A one-way line, kept open The source knows first and simply reports; the other end only listens. <i>Move:</i> the cheapest thing that beats asking repeatedly. Keep it open rather than reconnecting, and let the standard machinery do the reconnecting and position-tracking for you.	A private channel Both ends talk constantly, in a format only they need to understand. <i>Move:</i> take it knowingly. You have just become responsible for everything the intermediaries were doing — caching, compressing, labelling, and deciding what goes first.

The move that matters: the column is obvious and the row is the one that decides the bill. Every free service is provided by someone in the middle who can read the payload; the moment you make the channel opaque, all of that becomes your job. That is true of a network protocol, an encrypted file store, a private side-channel in an organisation, and a team that stops writing things down.

THE THUMB RULE, IN ONE SENTENCE

Count the round trips, then ask who in the middle is allowed to read the payload — the first number decides how fast it can be, and the second decides how much work you have just taken on.

It travels. A courier network can only reroute parcels it can identify. A shared document can only be indexed if it is readable. A conversation moved into a private channel is faster for the two people in it and invisible to everyone who might have answered it already. In each case the speed and the loss are the same decision, and the mistake is always the same one: choosing the private channel for traffic that other people were usefully handling for free.

Symptom → diagnosis → move

Cover the middle and right columns and work down the left.

Latency, bandwidth, and HTTP/1.x

WHAT YOU OBSERVE	MOST LIKELY DIAGNOSIS	WHAT TO DO
Doubling the link speed changed nothing.	You are spending round trips, not bytes — propagation delay, not transmission delay.	Reduce the number of round trips, or move the data closer with a CDN. Capacity is the wrong lever.
Distant users are several times slower for the same page.	Every round trip costs the propagation delay, and the page makes many.	Cut requests, cache aggressively, and serve from a nearby location. Expect 200–300 ms round trips at antipodal distances.
Latency became high and erratic after a network device change.	Bufferbloat — oversized buffers hiding the packet drops TCP relies on.	Active queue management. Note throughput tests will still look fine, which is what makes it confusing.
Tens of milliseconds gone before the first named hop.	Last-mile latency: 18 ms for fibre, 26 ms cable, 44 ms DSL at peak in one measurement.	Nothing you can fix from the server. Budget for it, and know it varies by technology, topology and time of day.
Requests are slow even though the server responds in 5 ms.	Fixed protocol overhead dominating: two round trips per request without connection reuse.	Turn on keepalive everywhere, including in your HTTP libraries, which often default to off.
Everything stalls behind one slow response.	Head-of-line blocking — HTTP 1.x cannot interleave response data.	Nothing within the protocol. Move the slow resource off the critical path, or onto a different connection.
The seventh parallel download hangs.	Six connections per host, with long-lived streams occupying some of them.	Move streams to a different hostname, reduce their number, or consolidate them into one.
Sharding made mobile slower.	Each shard costs a DNS lookup and a fresh TCP slow-start, and high-latency clients pay both hardest.	Reduce the shard count. Start from none and add only while measurements improve.
A 15-byte API response costs a kilobyte on the wire.	Uncompressed plain-text headers: 500–800 bytes before cookies.	Trim cookies first — they are the usual largest offender — and question every custom header.
Repeat visits got slower after bundling.	Lost cache granularity: one change invalidates the whole bundle.	Split a stable core from volatile code, and target roughly 30–50 KB compressed per script bundle.
The app is killed on older handsets.	A large sprite decoded and held whole in memory: width × height × 4 bytes.	Stop spriting large images. An 800 × 600 bitmap is about 1.83 MB regardless of the file size.
First render is late despite fast downloads.	Neither CSS nor JavaScript executes until the whole file has arrived.	Deliver first-paint-critical CSS separately and load the rest incrementally.
An inlined asset is being downloaded on every page.	Inlined resources are part of the document and cannot be cached separately.	Inline only small, page-specific, stable resources — roughly under 1–2 KB — and bundle the reused ones instead.

Streaming transports

WHAT YOU OBSERVE	MOST LIKELY DIAGNOSIS	WHAT TO DO
Updates arrive up to N seconds late, where N is your poll interval.	Queuing latency — the message was ready and waited for the next request.	A persistent connection: SSE for text one-way, WebSocket if the client must also send.
The stream works locally and dies or lags in production.	A buffering intermediary that does not understand the stream.	Deliver it over TLS so nothing in the middle can hold the bytes back.

Messages are missing after every reconnect.	The browser reports the last-seen ID; nothing on the server replays from it.	Assign event IDs and keep a server-side cache to retransmit from — or decide losses are acceptable and stop sending IDs.
Battery drain on mobile with a real-time feature.	Frequent small pushes waking the radio, often with unnecessary keepalives.	Batch messages. Remove application keepalives — the connection is not dropped because the radio went idle.
Binary data over a text stream costs a third more.	Base64 expansion, and a streamed asset that no cache can hold.	Push a notification and fetch the asset over HTTP. One extra round trip buys back caching and transfer encoding.
Small urgent messages stall behind a big one.	Head-of-line blocking on the socket: frames from different messages cannot interleave.	Split large messages, check <code>bufferedAmount</code> before sending, and keep your own priority queue.
<code>send()</code> returns instantly but data arrives late.	It only queues — a fast return is not delivery.	Monitor the buffered amount and send the next update only when it has drained.
Origin bandwidth far above forecast after moving to a socket.	Assets streamed over the socket are cached by nobody — not the browser, not a CDN.	Route cacheable payloads back over HTTP and keep the socket for control and updates.
A third of clients never connect.	Intermediaries that do not understand the protocol, or a network blocking it outright.	Use the secure scheme so the session is tunnelled, and keep a fallback for networks that block it entirely.
Connections die after about a minute.	Idle timeouts in your own proxy or load balancer, tuned for short bursty HTTP.	Raise them explicitly. On a load balancer, check the separate tunnel timeout — the handshake timeouts do not govern the upgraded connection.
The fallback transport is far slower than expected.	The polyfill defaults to polling or long-polling.	Check what the library actually falls back to, and hold it to the same performance goals as the native path.
The receiver cannot tell what a payload is.	There is no per-message metadata: one bit distinguishes text from binary.	Negotiate a subprotocol, and carry your own header inside the message or in a preceding text message.

Reverse mapping — you see a technique, what is it there for?

The other direction. Use this when auditing a design rather than diagnosing an incident: for each mechanism you find, say what it was buying and what it does *not* buy.

WHAT YOU FIND IN THE DESIGN	WHAT IT COUNTERS	WHAT IT DOES <i>NOT</i> COVER
A content delivery network	Propagation delay — the data is physically closer, so every round trip is shorter.	The number of round trips. A chatty page is still chatty; it is just chatty over a shorter distance.
Keepalive on every client and proxy	The handshake round trip and repeated slow-start: $(N-1) \times RTT$ across N requests.	Head-of-line blocking. Responses still cannot interleave, so one slow response still blocks the queue.
Six connections, or domain sharding	The absence of multiplexing — it overlaps the waiting instead of removing it.	The cost per request or the number of them. And each shard adds a DNS lookup and a fresh slow-start.
Concatenation and spriting	Per-request protocol overhead, and the lack of pipelining — the bytes arrive back to back.	Cache granularity, sprite memory, or execution latency; it makes all three worse.
Inlining under a kilobyte or two	The request entirely, for resources smaller than their own HTTP overhead.	Reuse across pages. An inlined asset cannot be cached separately, so it ships with every page that inlines it.

Gzip on text assets	Transferred bytes — typically a 60–80% reduction for one configuration flag.	Round trips, and anything already compressed. It is a bytes lever, not a latency lever.
Cache headers and revalidation	The request itself — the only technique here that costs nothing anywhere else.	The first visit, and anything genuinely dynamic. Also nothing, if the resource is inlined into a document.
Server-Sent Events	Queuing latency on server-to-client text, at about 5 bytes a message, with reconnection handled for you.	Propagation latency, client-to-server data, binary payloads, and one of your six connections per host.
Event IDs on a stream	The client's ignorance of where it left off — it will tell you on reconnect.	The replay itself. Without a server-side cache the ID is reported and nothing is resent.
A WebSocket	Queuing latency in <i>both</i> directions, per-message overhead, and the text-only limit.	Caching, compression, metadata, and head-of-line blocking — which returns, now driven by your own message sizes.
The secure scheme for a socket or stream	Intermediaries that buffer, modify, or refuse to upgrade the connection.	Idle timeouts. Anything in the path can still drop a quiet TCP connection, including your own infrastructure.
Subprotocol negotiation	The two ends silently disagreeing about what the opaque bytes mean.	Per-message metadata. Beyond selecting a name, it has no effect — you still carry your own headers inside messages.
Masking on client frames	A malicious script poisoning caches in intermediaries that do not understand the protocol.	Confidentiality. It is not encryption, and it does nothing about a transparent proxy modifying data.
A polyfill or real-time framework	Browsers without native support, and often reconnection and heartbeat logic too.	Performance. The fallback transport — usually polling — decides what you actually get.

Numbers worth remembering

NUMBER	WHAT IT IS	WHY IT MATTERS IN AN ARGUMENT
~200,000 km/s	The speed of light in optical fibre — a refractive index of about 1.5, against 299,792 km/s in a vacuum. Real fibre is 1.4 to 1.6.	The single most useful number here. It converts any distance into a floor nobody can negotiate below.
56 ms · 160 ms	Round trip in fibre along the great circle: New York to London (5,585 km) and New York to Sydney (15,993 km).	Two anchors. Quote them when someone proposes a design that needs several round trips per interaction.
200–300 ms	The <i>actual</i> New York to Sydney round trip over existing networks, against 160 ms on the great circle.	The gap between physics and cables. Never quote the theoretical number as the expected one.
100–200 · 300 · 1,000 ms	Perceptible lag · described as sluggish · the user has mentally moved on.	Turns a latency budget into a product requirement, which is the only form anyone acts on.
18 · 26 · 44 ms	Peak-hour last-mile latency for fibre-to-the-home, cable and DSL in one regulator's measurements.	Tens of milliseconds before the packet reaches the provider's core. Budget for the last mile, not just the ocean.
~30%	The most that better materials and faster routers can improve latency, given we are within ~1.5× of light speed.	Ends the “the network will get faster” argument for latency, while leaving it open for bandwidth.
10 s vs 0.1 s	Time to put a 10 Mb file on the wire over 1 Mbps and over 100 Mbps.	The cleanest demonstration that transmission delay is about link rate and has nothing to do with distance.
2 round trips	The minimum cost of one HTTP request over a new TCP connection: handshake, then request and response.	The fixed cost keepalive removes. The faster your server, the more this dominates.
(N–1) × RTT	The total latency saved by connection reuse across N requests.	With 90-plus resources on an average page, the saving is measured in seconds. The best-value line in this booklet.
284 → 228 → 172 ms	The same two-resource fetch with separate connections, with keepalive, and with pipelining — a 40% reduction.	Shows protocol choices alone moving a real number, and that the saving grows with latency and request count.
6 per host	Connections most browsers open to a single origin.	Explains both the parallelism you get and the seventh request that hangs — including when your own streams hold the slots.
500–800 bytes	HTTP metadata on every browser-initiated request, before cookies, always plain text.	Set against a 15-character JSON body wrapped in 352 bytes of headers — 96% overhead — it makes the case for trimming headers and cookies.
60–80%	Typical size reduction from gzip on text assets.	The highest benefit-to-effort ratio available: usually a configuration flag.
33%	Byte expansion from base64 encoding, which turns bytes into ASCII at a 4/3 ratio.	The tax on inlining binary assets and on pushing binary through a text-only stream.
1–2 KB · 30–50 KB	Rule-of-thumb ceiling for inlining a resource · target size for a compressed script bundle.	Two thresholds that settle most bundling arguments without further measurement.
1.83 MB	Memory for an 800 × 600 decoded image: width × height × 4 bytes, held whole.	Why spriting can be the thing that kills the app on an older handset, regardless of the file's size on disk.
~5 bytes	Minimum per-message overhead of an SSE event.	The reason a text stream is cheaper than anything built on repeated requests.

2–3 seconds	The recommended and commonly implemented default SSE reconnect delay, overridable by the server.	Tells you roughly how large a replay gap to expect, and therefore how far back the server cache must reach.
2–10 · 6–14 bytes	WebSocket framing overhead: server-sent frames, and client frames with their 4-byte masking key.	Two orders of magnitude below an HTTP 1.x request. The concrete case for a socket on chatty traffic.
0–125 · 126 · 127	The payload-length field: inline, then 2 further bytes for a 16-bit length, then 8 for a 64-bit length.	Explains why the frame header is variable-length, and where the 2-to-10-byte range comes from.
1 · 2 · 8 · 9 · 10	WebSocket opcodes: text, binary, close, ping, pong.	Enough to read a frame dump, and to know that liveness checks are in the protocol rather than your application.
60–100 bytes	Combined IP, TCP and TLS framing per message, whatever the application protocol.	The floor under every overhead comparison in this booklet — worth quoting when a few bytes of framing are being argued over.
60 seconds	A common proxy default timeout for an idle connection, against the hour a long-lived session may need.	The first place to look when sockets and streams die at suspiciously round intervals.

Glossary — the terms you should be able to define cold

TERM	DEFINITION, AND THE LINE THAT SEPARATES IT FROM ITS NEIGHBOUR
Latency / bandwidth	Time for a packet to arrive / maximum throughput of the path. Different quantities with different economics; for most pages, latency is the bottleneck.
Propagation delay	Distance ÷ signal speed. The only irreducible delay; the only lever is less distance.
Transmission delay	Packet bits ÷ link rate. Independent of distance, and the delay more bandwidth actually fixes.
Processing / queuing delay	Time being handled by a router / time waiting to be. Both are per hop; the second grows with load.
Bufferbloat	Oversized buffers hiding the packet drops TCP uses as a congestion signal, producing high and variable delay.
Refractive index	Speed of light in a vacuum ÷ speed in the medium. About 1.5 for fibre; larger means slower.
Great-circle path	The shortest route between two points on the globe. A lower bound, not a route that exists.
Last-mile latency	Delay in the first hops to your provider's core. Often larger than the intercontinental portion.
Wavelength-division multiplexing	Many channels of light in one fibre. Why capacity grows without new cable being laid.
Keepalive	Reusing one TCP connection for several requests. Saves $(N-1) \times \text{RTT}$; enabled by default in HTTP 1.1, often off in libraries.
Pipelining	Sending requests without waiting for responses, moving the queue to the server. Not multiplexing, and effectively dead in browsers.
Head-of-line blocking	One slow item blocking everything behind it. Happens in HTTP 1.x responses, in TCP on packet loss, and in WebSocket messages.
Multiplexing	Interleaving data from several responses or streams on one connection. The feature whose absence generates Part 2.
Domain sharding	Spreading resources over hostnames to multiply the per-host connection limit. Costs a DNS lookup and a slow-start each.
Concatenation / spriting	Bundling scripts or images into one resource. Application-layer pipelining, paid for in cache granularity and memory.
Resource inlining	Embedding a resource in the document. Removes the request; removes separate cacheability with it.

Server-Sent Events	Server-to-client text streaming over one long-lived HTTP response. Text only, one direction only.
Event stream format	data, event, id and retry fields, newline-delimited, blank line ends the event. That is the whole protocol.
Last-Event-ID	The header the browser sends on reconnect, carrying the last ID it saw. Reporting, not replay — the replay is yours.
Queuing latency	Time a ready message waits before it can be sent. What streaming removes; propagation latency is untouched.
WebSocket	Bidirectional, message-oriented text and binary streaming, negotiated by HTTP Upgrade. Not a superset of HTTP.
Frame / message	The wire unit with a 2–14 byte header / the complete logical message your callback receives.
Masking	Client-to-server payload obfuscation with a 32-bit key, against cache poisoning of naive intermediaries. Not encryption.
Subprotocol negotiation	Client advertises names, server picks one, or the connection is refused. The only metadata channel a socket has.
Protocol extension	New opcodes and fields in the framing layer, negotiated at handshake. Multiplexing and compression were the two.
bufferedAmount	Bytes queued but not yet sent. The check that stops one large message head-of-line blocking the urgent ones.

Answer key

PART 1 · Q1 — THE SYDNEY DASHBOARD AND THE UPLINK UPGRADE

(a) The Sydney figure is dominated by **propagation delay**, which is distance divided by signal speed and has nothing to do with anyone's link capacity. Upgrading the origin uplink reduces **transmission delay** — the time to push bits onto the wire — which is a function of the link's data rate and is independent of distance. Those are different delays with different causes, and the evidence in the question already rules out saturation: London, on the same origin and the same uplink, is fine. If the origin were saturated both would be slow. The remaining two delays, **processing** and **queuing**, also grow along the longer route because there are more hops, each reading headers and each with its own buffer.

(b) The floor: New York to Sydney is 15,993 km and gives 160 ms round trip along the great circle, and the real figure over existing cables is **200–300 ms**; London to Sydney is a comparable order of magnitude. Take a round trip of roughly 250 ms. The arithmetic is distance ÷ ~200,000 km/s for the one-way time in fibre, doubled — noting that fibre is about 1.5× slower than light in a vacuum. The 78 requests are what turn that into 4.2 seconds: **each round trip is spent serially unless the requests overlap**, and with six connections per host and no multiplexing they overlap only six at a time. That is roughly thirteen sequential batches of 250 ms before you count server time — which lands in the right neighbourhood. Credit for observing that users are already past the 1,000 ms mark at which they mentally context-switch.

(c) Two structurally different fixes. **Reduce the number of round trips** — fewer resources, caching so repeat visits skip requests entirely, bundling, keepalive — which attacks how many times you pay the propagation delay, not the delay itself. And **move the data closer** with a CDN, which attacks the propagation delay directly and is the only lever on it. The thing that would genuinely lower the floor is **a shorter cable along the great circle** — and nobody does it here because laying cable is constrained by terrain, politics and cost, and is only justified where milliseconds are worth millions. Also creditable: better fibre and faster routers would help, but we are within about 1.5× of light speed so the ceiling on that strategy is roughly 30%.

PART 1 · Q2 — THE GATEWAY THAT NEVER DROPS A PACKET

(a) **Bufferbloat**. The new gateway ships with large incoming buffers on the assumption that dropping packets should be avoided at all costs. When arrivals outpace service, packets are queued rather than dropped, and the time they spend in that buffer is **queuing delay** — which is high and, critically, **variable**. Throughput looks fine because bufferbloat is not a capacity problem: every byte still arrives, and a throughput test measures exactly that. It measures the wrong quantity for this fault.

(b) Because TCP's congestion avoidance **uses packet loss as its signal**. A dropped packet is how the network tells a sender to slow down. A buffer that absorbs the overflow suppresses that signal, so senders keep increasing their rate while the queue grows, and instead of a prompt backoff you get a long and unpredictable queue in front of every packet. The design goal is well-intentioned and the mechanism it breaks is the one that was keeping latency bounded — which is why the fix is not a bigger or a smaller buffer but a smarter one.

(c) The class of solution is **active queue management** — algorithms that deliberately manage queue delay rather than maximising buffer occupancy; the CoDel algorithm was proposed for exactly this and is available in modern kernels. Moving the video server closer would not fix it because a CDN reduces **propagation** delay, and the delay here is **queuing** delay being introduced by a device inside the office, after the packet has already arrived. Shortening the journey does nothing about a queue at the destination.

PART 2 · Q1 — THE DASHBOARD WHOSE FOURTH CHART NEVER LOADS

(a) The browser opens at most **six connections per host**, and the two event streams and the long-lived polling request occupy three of them permanently. With the chart requests competing for what remains, the seventh outstanding request to that host simply **hangs until one of the earlier ones completes** — which is why closing a panel unblocks it. Idleness is irrelevant because HTTP 1.x has **no multiplexing**: a connection carries one exchange at a time whether or not any bytes are moving, so a stream that has sent nothing for ten minutes is still holding a full TCP stream.

(b) The underlying limitation is the absence of multiplexing, which is why browsers open several connections at all — and the per-host cap exists as a **safety check**, so that a client cannot saturate all available resources on a server, intentionally or otherwise. The irony is that the same check enables the reverse: exceed it on the client and all further requests to that host are blocked. Applications built on real-time delivery — sockets, event streams, hanging requests — are exactly the ones that hit it, so you can create a **self-imposed denial of service** against your own application.

(c) Two structurally different fixes. **Consolidate the streams**: carry all the update types over one connection, distinguishing them by event type, so you spend one slot instead of three — costing you a shared failure domain and some multiplexing logic in the application. Or **move the streams to a different hostname**, so they draw against a separate connection pool — costing an extra DNS lookup, a fresh TCP slow-start, and possibly a fresh TLS handshake, all of which hit high-latency clients hardest. Also creditable: reduce the number of long-lived connections generally, and do not add the fifth.

PART 2 · Q2 — THE BUNDLE THAT HELPED DESKTOP AND HURT EVERYTHING ELSE

(a) Three distinct trades. **Mobile got worse** because a 900 KB bundle must arrive in full before *any* of it executes — neither CSS nor JavaScript allows incremental execution — so on a high-latency, low-bandwidth link the time to first render is dominated by a transfer that used to be able to start executing in pieces. **Repeat visits got worse** because all 40 modules now share one cache key: a single change to any one of them invalidates and re-downloads the whole 900 KB. That is the loss of **cache granularity**, and it is the classic cost of bundling. **The app is being killed** because a sprite is decoded and held in memory as a bitmap at **width × height × 4 bytes**, regardless of the compressed file size and regardless of how little of it is on screen — the browser does not clip the unused part out of memory.

(b) **No, and it may make things worse**. Sharding multiplies the per-host connection limit; it does not reduce bytes, does not restore cache granularity, and does not reduce memory. It cannot even parallelise *this* download, because one 900 KB resource is a single request and lives on a single connection. What it does add is a DNS lookup and a fresh TCP slow-start per shard, which penalises exactly the high-latency mobile clients that are already the problem.

(c) Split the bundle: separate the **stable core** — frameworks and libraries that rarely change — from volatile application code, so a routine change invalidates only a small bundle; target roughly **30–50 KB compressed** per script bundle, large enough to amortise per-request overhead and small enough to allow incremental execution; deliver first-paint-critical CSS separately from the rest. Stop spriting the icons at that scale, and inline only the small, page-specific ones — roughly under 1–2 KB, remembering base64 costs 33% and an inlined asset cannot be cached separately. What remains true regardless of protocol: **eliminate unnecessary latency and minimise transferred bytes**. Bundling and sharding are workarounds for a protocol that cannot interleave responses; caching, compression and deleting resources are not.

PART 3 · Q1 — THE PRICE STREAM WITH A GAP AFTER EVERY RECONNECT

(a) On reconnect the browser does two things automatically: it **reestablishes the connection** after the retry interval, and if the server has been assigning IDs it sends the last one it saw back as a **Last-Event-ID** header. That is the whole of the browser's contribution. **Replaying the missed messages is entirely the server's job** — it must maintain a local cache of recent events and, on seeing that header, retransmit everything after that ID. A gap after every reconnect means either the server is not sending IDs at all, so the client has nothing to report, or it is sending them and ignoring the header on reconnect. Either way the browser is behaving exactly as designed.

(b) A reconnect cycle at a suspiciously round 60 seconds points at an **idle or read timeout in the infrastructure** — a proxy, load balancer or server tuned for short bursty HTTP rather than long-lived streams — or at an intermediary buffering the stream. The heartbeat is the wrong fix because it treats a configuration problem as a protocol problem: it does not raise anyone's timeout, it merely keeps traffic flowing so the timeout is not reached, which is a workaround that has to keep working forever. Its cost is paid by the mobile clients: a message every 5 seconds **wakes the radio** continuously, which is exactly the real-time push anti-pattern that drains handset batteries, and an SSE connection is not dropped merely because the radio has gone idle. The right sequence is: raise the timeouts you control, put the stream over TLS so intermediaries cannot buffer it, and batch real messages instead of manufacturing filler ones.

(c) Stop base64-encoding the charts into the stream. The stream is **UTF-8 only** by design, so binary costs a **33%** byte expansion, and a streamed asset **cannot be cached by the browser cache at all**. Instead use the stream as a notification: push a small text message saying a new chart exists and where, and let the application fetch it with an ordinary request. The cost is **one extra round trip** per image. What it buys back: the asset becomes cacheable by the browser and by any intermediary, and it gets ordinary transfer encoding and compression — plus the 33% goes away. Also creditable: gzip the event stream itself, which is legitimate because it is an ordinary HTTP response.

PART 3 · Q2 — CHOOSING A PUSH TRANSPORT FOR JOB PROGRESS

(a) The claim is wrong as stated. Changing transport **does not decrease the round trip** between client and server at all — propagation latency is a function of distance and the speed of light and is identical for polling, a stream or a socket. What both persistent transports eliminate is **queuing latency**: with 5-second polling, a status line can be ready on the server and still wait up to 5 seconds for the next request, whereas a persistent connection lets the server dispatch it the moment it exists. SSE and WebSocket are **identical** on this axis for server-to-client data; the socket adds nothing here. It only differs for messages travelling the other way.

(b) Three things SSE gives free that a socket does not. **Automatic reconnection**, at a default of 2–3 seconds and adjustable by the server with a retry field. **Last-seen message tracking**, where the browser remembers the ID and reports it on reconnect. And **message parsing with no unbounded buffer** — the browser finds the boundaries, extracts the payload and fires a DOM event per message, discarding what it has processed. Also creditable: session-wide gzip, which works because the stream is all text and an ordinary HTTP response; and the fact that it is ordinary HTTP, so it deploys through existing infrastructure with no new negotiation.

(c) The report should be delivered as a **notification on the stream plus an ordinary HTTP request** to fetch it — one extra round trip, in exchange for a cacheable, compressible, resumable download that any intermediary can serve, and no base64 tax. The WebSocket becomes right when the **client** needs to send frequently and with low latency — live cancellation, parameter changes during the run, a cursor or control channel — or when the downward data becomes genuinely binary and high-volume. Reaching for it merely because the client *might* send later trades caching, compression, reconnection and last-ID tracking for a capability nobody is using yet.

PART 4 · Q1 — THE MAP THAT FREEZES WHEN YOU PAN

(a) **Head-of-line blocking on the socket.** Messages are delivered in exactly the order the client and server queue them, and although a message may be split into several frames, **frames from different messages cannot be interleaved** — there is no stream identifier in the framing layer. So a large tile message occupies the connection until it has been fully sent, and every cursor update queued behind it waits. A bigger send buffer makes it worse rather than better: it lets more large messages queue up ahead of the small ones, which is more of the same problem. The fix is at the application layer — split large messages, check what is already queued, and prioritise.

(b) The tiles stopped being **cacheable**. Delivered as ordinary HTTP resources they are held by the browser cache, by any intermediary proxy, and by a CDN, so a panned-over tile is fetched once and a popular tile is served to thousands of users from an edge node. Streamed inside a custom protocol they are opaque: nothing in the middle can identify or store them, so every client fetches every tile every time and all of that traffic terminates at the origin. Roughly ten times the forecast is exactly what losing a cache hierarchy looks like.

(c) Route by cacheability. **Cursor positions, edits, presence and control messages go over the socket** — genuinely non-cacheable, latency-sensitive, small, and needed in both directions. **Tiles go over ordinary HTTP**, fetched by the client, where they are cached by the browser and the CDN and compressed by ordinary negotiation; the socket can carry a message telling the client which tiles to fetch. The client-side check the application should be doing regardless is to **monitor bufferedAmount** before sending: `send()` is asynchronous and returns immediately, so a fast return is not evidence of delivery, and sending the next update only once the buffer has drained is what keeps a backlog from forming.

PART 4 · Q2 — A THIRD OF CLIENTS NEVER CONNECT

(a) The most likely cause is **intermediaries that do not understand the protocol** — explicit and transparent proxies, firewalls and carrier equipment between the client and the server. Because the connection begins as an HTTP Upgrade, such devices may blindly upgrade without understanding what they upgraded to, buffer frames, modify content, or misclassify the traffic as a compromised HTTP connection and drop it. Mobile clients are worst affected because their traffic crosses the most such equipment. The single change that addresses most of it is to **use the secure scheme**: negotiating TLS before the Upgrade handshake creates an end-to-end tunnel that intermediaries cannot inspect or alter, and in practice it significantly raises the success rate of establishing the session. Some networks block this traffic outright, which is why a fallback is still needed.

(b) Deaths at about a minute are **idle timeouts in infrastructure the team operates**. HTTP is optimised for short bursty transfers, so proxies, load balancers and servers are routinely configured to time out idle connections aggressively — one widely used proxy defaults to 60 seconds — and that is exactly wrong for a long-lived session. Raise the read and write timeouts explicitly on the proxy. The setting most often missed is on the **load balancer**: its connect, client and server timeouts apply only to the *initial HTTP Upgrade handshake*, and once the upgrade completes the connection is governed by a separate **tunnel** timeout that has to be raised on its own. Note also that TLS does not help here — it improves the odds of establishing the session but does not stop anyone timing out an idle TCP connection.

(c) The polyfill is a performance risk because the hard part of emulation is not the API but the **transport**: the fallbacks are polling, long-polling or a streaming request, and the library's choice decides what a fallen-back client actually experiences — polling reintroduces both the queuing latency and the full per-request HTTP overhead the socket existed to remove. Always prefer the native interface and hold the fallback to the same performance goals. The key-and-accept exchange in the handshake is a **security policy against blind upgrades**: it proves the server genuinely understands the requested protocol version rather than having switched protocols without knowing what it switched to. It does **not** authenticate anyone, does not encrypt anything, and does not protect against a **transparent proxy** that analyses and modifies data on the wire without announcing itself — which is precisely why the answer to (a) is a tunnel.

HOW TO SCORE YOURSELF

Two marks, both quick. **Completeness**: did you name every element, and — for this competency especially — did you name the *delay* or the *service* at stake? “It’ll be faster” is not an answer; “it removes queuing latency and leaves propagation unchanged” is. **Phrasing**: could you have said it in a design review without hand-waving? If you wrote “caching stops working” where the key says “the payload is opaque, so neither the browser cache nor any CDN can hold it and all the traffic lands on origin”, the label is there and the mechanism isn’t — and the mechanism is what changes anyone’s mind.

Spaced-review tracker

Review at **day 1, 3, 7, 16, 35**. Write the date in the box when you pass. On a fail, reset that row to a 1-day interval and start again.

ITEM	DAY 1	DAY 3	DAY 7	DAY 16	DAY 35	FAILS
1 • Latency vs bandwidth, and why the economics differ						
2 • The four delays, each with what it depends on						
3 • Speed of light in fibre, and the refractive index						
4 • Great circle vs the cables that exist						
5 • Bufferbloat, and why avoiding drops backfires						
6 • Last-mile latency, and the ~30% ceiling						
7 • The two-round-trip fixed cost of a new connection						
8 • Keepalive: $(N-1) \times \text{RTT}$, and why N matters						
9 • Pipelining, and the four reasons it failed						
10 • Head-of-line blocking in HTTP 1.x						
11 • Six per host, and the self-inflicted denial of service						
12 • Domain sharding, and what each shard costs						
13 • Protocol overhead: 500–800 bytes, plus cookies						
14 • Bundling and spriting: the four costs						
15 • Inlining, the 1–2 KB rule and the 33% tax						
16 • Workaround vs evergreen optimisation						
17 • SSE on the wire: the four fields						
18 • What the browser does for an event stream						
19 • Last-Event-ID: reporting versus replay						
20 • SSE's two hard limits, and the doorbell pattern						
21 • Queuing latency vs propagation latency						
22 • The Upgrade handshake and the 101 response						
23 • Frame vs message, and the header fields						
24 • Masking: what it defends, and what it doesn't						
25 • Framing overhead against SSE and HTTP 1.x						
26 • Subprotocol negotiation, and the no-match case						
27 • The services a socket stops receiving						
28 • bufferedAmount, and blocking returning						

29 • TLS, idle timeouts, the tunnel setting						
30 • Master matrix — direction × readability						

Final quiz — no notes, twenty minutes, write the answers

- Name the four components of latency, say what each is a function of, and name the only one that more bandwidth reduces.
.....
.....
- Explain why latency and bandwidth have different futures, giving the physical reason and the approximate ceiling on improving latency.
.....
.....
- Explain bufferbloat: what the designers were trying to achieve, what mechanism it breaks, and why a throughput test looks fine.
.....
.....
- Give the latency saving from connection reuse as a formula, and say why the fixed cost of a new connection matters more the faster your server is.
.....
.....
- Define head-of-line blocking in HTTP 1.x, and give three distinct reasons pipelining was abandoned in browsers.
.....
.....
- Explain why six connections per host can produce a denial of service against your own application, and why an idle stream is not free.
.....
.....
- Name one distinct cost each for domain sharding, bundling and inlining, and say which of the three would still make sense under a protocol that could interleave responses.
.....
.....
- Describe the Server-Sent Events wire format and say exactly which parts of the reconnect-and-recover story the browser handles and which you do.
.....
.....
- State which kind of latency a persistent connection removes and which it does not, then give SSE's two hard limits and the standard workaround for the second.
.....
.....
- Distinguish a WebSocket frame from a message, and name three services you stop receiving once the connection has been upgraded.
.....
.....